

Block 1: Framing

Block 1 · 0:00 – 0:20 · 20 minutes

For this block we're going to **listen to why analytics agents fail differently than coding agents** in order to **understand the problem we're solving today**.

What's happening

Rob presents the core idea: coding agents worked first because code has compilers and tests — if the code is wrong, something breaks. Analytics has no compiler. One correct answer, many plausible wrong ones, and nothing flags the difference. That's the gap skills fill.

What you're creating

Nothing yet. This is context-setting. You're watching, not working. The hands-on starts in 20 minutes.

What to pay attention to

The specific claim being made: an AI agent can give a confident, precise, well-formatted answer that is silently wrong — and the only way to catch it is to have encoded what "right" looks like before the agent ran.

If you have a question, try framing it as multiple choice:

"Is the problem that the agent picks the wrong table, that it picks the wrong rows, or that it doesn't know which definition to use?" is easier for everyone to learn from than "I don't get it."

Block 2: Failure Demo

Block 2 · 0:20 – 0:40 · 20 minutes

For this block we're going to **watch an AI agent answer the same question twice** — once with no instructions and once with a skill loaded — in order to **see the difference a reference doc makes before we build one ourselves.**

What's happening

Rob asks the agent: "How many employees do we have? Just give me the number, I'm in a hurry." The agent answers confidently both times, but the two answers look very different. One gives a bare number with nothing to check. The other names its source, states its assumptions, and flags what it's uncertain about.

What you're creating

Nothing yet. Still watching. But notice *what specifically* made the first answer worse — you'll use those observations as raw material in the Validate block.

What to pay attention to

- Did the first answer name where the number came from?
- Did it say who was counted?
- Did it say what it assumed?
- What changed in the second answer — and could a reader tell?

If you have a question, try framing it as multiple choice:

"Was the second answer better because it used a different table, because it added a footer, or because it clarified the population?" gives you a concrete answer.

Block 3: Define

Block 3 · 0:40 – 1:00 · 20 minutes

For this block we're going to **specify a question before looking at any data** and create a **Question Spec** in order to **pin down what we're actually asking — and what we're assuming — before the numbers bias us.**

What's happening

You pick one of three domains (Attrition, Compensation, or Engagement), take its seed question, and fill in: who counts, what time period, what measure, what comparison, and what you're assuming. This choice carries through the rest of the day.

What you're creating

A completed Question Spec — a structured document that names every assumption behind your question. The most useful field is the last one: *Open Assumptions* — the things you resolved by guessing rather than confirming.

What you need

- The **Define Worksheet** (printed or on screen)
- The **Data Dictionary** printout — your map of the warehouse

If you have a question, try framing it as multiple choice:

"Should I define 'attrition rate' as voluntary only, all separations, or regrettable only?" lets Rob help you fast. "I'm not sure what to write" takes longer to untangle.

Block 4: Validate

Block 4 · 1:00 – 1:30 · 30 minutes

For this block we're going to **write the test before we write the answer** and create an **eval file** (`evals/my-eval.json`) in order to **define what "good" looks like before the data can bias our judgment.**

What's happening

You write 2–3 assertions — specific, checkable statements about what a good answer to your question should include. Then you predict whether a plain agent with no instructions would pass or fail each one. You haven't queried the database yet, and that's by design.

What you're creating

A JSON file saved as `evals/my-eval.json` containing your question, a description of a good answer, and your checklist of assertions. This file is what the ablation script will use to score your reference doc later.

What you need

- The **Validate Worksheet**
- Your completed **Question Spec** from Define
- If stuck on the JSON format: ask Rob for the **Example Evals** handout

If you have a question, try framing it as multiple choice:

"Is my assertion too vague, too specific, or checking the wrong thing?" gets you a concrete fix. "Is this right?" gets you a shrug.

Block 5: Build

Block 5 · 1:30 – 2:15 · 45 minutes

For this block we're going to **query the warehouse and discover what it gets wrong** and create a **reference doc** (`references/<domain>.md`) in order to **teach the agent what an experienced analyst would already know about this data**.

What's happening

Now you open the database. You run queries, find discrepancies, and write them up. The Gotchas section — where you document two plausible answers that disagree — is the core exercise and gets the most time (15 minutes).

What you're creating

A **reference doc saved as `attrition.md`, `compensation.md`, or `engagement.md`** inside the `references/` folder. This is the document the ablation script will load alongside the agent to see if your Gotchas, filters, and query patterns actually improve its answers.

What you need

- The **Build Worksheet**
- The **Data Dictionary** — your column and table reference
- Your **explore.py** file set up and running from the terminal
- If stuck on SQL: ask Rob for the **Starter Queries** handout

If you have a question, try framing it as multiple choice:

"Am I getting the wrong number because I'm using the wrong table, missing a filter, or double-counting from a join?" narrows the problem to one fix. "My number looks wrong" doesn't.

Block 6: Ablation

Block 6 · 2:15 – 2:35 · 20 minutes

For this block we're going to **run the same question with and without our reference doc** and create **a measured comparison** in order to **find out if what we built actually helped — and check our Validate predictions.**

What's happening

You run one command. The script asks your question twice — once with no reference doc (baseline), once with yours loaded — and scores both against the assertions you wrote in Validate. Then you compare the result to your prediction.

What you're creating

A side-by-side result: baseline score vs. with-doc score, plus a comparison against the prediction you wrote 45 minutes ago. Were you right about which assertions would fail?

What you need

- Your `evals/my-eval.json` (from Validate)
- Your `references/<domain>.md` (from Build)
- Your **Validate Worksheet** with your predictions written on it
- A terminal open in the repo folder

The command

```
cd workshop
python run_my_ablation.py
```

If you have a question, try framing it as multiple choice:

"Did my baseline fail because my assertion is too strict, because the agent genuinely missed it, or because my JSON has a typo?" tells Rob exactly where to look.

Block 7: Operationalize

Block 7 · 2:35 – 2:50 · 15 minutes

For this block we're going to **learn how a reference doc stays alive after today** in order to **understand the difference between a one-time fix and a system that improves over time.**

What's happening

Rob covers three ideas: the provenance footer (how every delivered answer discloses its source and assumptions), correction harvesting (turning stakeholder corrections into new Gotchas and assertions), and freshness checking (how to catch a doc that's gone stale).

What you're creating

Nothing — this is a talk-through block. You're connecting what you built today to how it would actually work in a production analytics environment. No worksheet, no file to save.

The key takeaway

A reference doc written once and never updated is worse than no doc at all — it becomes a confident, authoritative source of wrong answers. The correction loop (stakeholder says you're wrong → you add a Gotcha → you add an assertion → the next run catches it) is what turns a static document into a system that learns.

If you have a question, try framing it as multiple choice:

"Does correction harvesting happen automatically, does someone have to do it manually, or does the agent do it itself?" gets you a clear answer.

Block 8: Contribute and Wrap

Block 8 · 2:50 – 3:00 · 10 minutes

For this block we're going to **contribute our reference doc to the open-source repo** and create a **pull request** (or send an email) in order to **make today's work available to everyone who uses this pack after us**.

What's happening

The repo is real, public, and MIT-licensed. Your reference doc is a real contribution. You can open a pull request from your browser in about 8 clicks — no git commands needed. If you're not comfortable making a GitHub account, emailing your file to Rob works just as well.

What you're creating

- | **A pull request** (or an email attachment) containing your references/<domain>.md.
- | When it's merged, your name goes in the README's Contributors section.

What you need

- A GitHub account (free — sign up at github.com if you don't have one)
- Your completed reference doc from Build
- The **PR Flow Cheat Sheet** if you've never opened a pull request before

| **If you have a question, try framing it as multiple choice:**

| "Do I upload to the main page, the references folder, or my own fork?" is answerable in one sentence. "How does GitHub work?" is a workshop unto itself.